

P?S/8 Tiny File System Internal Directory Structures

Last edit: 15-Jul-2017 cjl

Note: Information regarding other P?S/8 file structures such as the Extended-length File Directory Entry and the System Programs Directory in the basic P?S/8 system, or the directory structure used in the SHELL overlay can be obtained from other support documents covering these structures.

The basic P?S/8 file structure for user-created files is the Tiny File System or *TFS* directory. The P?S/8 keyboard monitor (and embedded line editor) includes a variety of commands to edit text files stored in the TFS directory, as well as additional commands to manage files within the directory (regardless of format). All supported functions can be accomplished without running any system programs; as such, this component of the P?S/8 operating system is a fast and responsive environment for primary management of all TFS-based user files. For additional documentation of the P?S/8 keyboard monitor consult the *P?S/8 Keyboard Monitor Command Guide* document available separately.

Note: There are additional P?S/8 system programs available to perform certain tasks by alternate means; however, there are tradeoffs associated with using programs that are oriented around only certain aspects of file management without providing any means to accomplish other functions found in the keyboard monitor or in other utility programs. Users familiar with the keyboard monitor environment can generally examine files and update the TFS directory to carry out file-related functions more quickly than running other utilities. For example, the keyboard monitor *CA{talog}* command supports no particular options (other than the ability to change the default logical device unit which is a feature of all relevant keyboard monitor commands), yet produces virtually instantaneous results. For a comprehensive set of potential directory printout options, the *DIRECT{ory}* program can be used as needed; for more complex directory management operations potentially addressing multiple files via a wild-card filter mechanism, the *FILMAN* (File Manager) program is available. These and other utilities must be initiated from the keyboard monitor; the system must reload after each program is executed before any other operation can be accomplished. Depending on hardware specifics, this method, while clearly offering many additional options, may require considerably longer time to accomplish general file and directory navigation tasks.

All TFS files are exactly 2048 12-bit words long stored in 16 consecutive logical blocks of 128 12-bit words each. The P?S/8 keyboard monitor requires valid internal parameters within the TFS directory to allow proper operation; there are specific requirements for valid file name characters within file slots.

TFS file names can be as long as six characters where each character is either a letter, number or certain specific non-alphanumeric characters such as `\`, `^`, `[`, and `]`. Some users prefer to organize a particular programming project by using file names suggestive of pseudo-sequence numbers such as `PRG[1]`, `PRG[2]`, `PRG[3]`, `PRG[4]`, `PRG[5]`, `PRG[6]`. (Note: All P?S/8 system programs can be passed a maximum of 17 files; specific programs may impose further file restrictions. For example, P?S/8 *PAL* will generally require binary output files which somewhat limits the maximum practical number of input files; virtually all system programs allow some combined total of 17 input and output files.)

The internal entry for any TFS file always occupies three adjacent 12-bit words on a three-word boundary in sequence within blocks 0015-0016. Figure 1 illustrates a few file entries in a minimally-used directory.

Starting with an empty directory, the files `TCBIN`, `TCBOOT`, `FOO` and `BAR` were entered, then `FOO` was deleted creating a free slot.

Note: A file slot not currently assigned due to deletion is referred to as a *free* slot. Empty slots after the last occupied slot are referred to as *available* slots. While there is no practical difference between the two, free slots are not the highest in the directory as the storage is laid out in linear order. Whenever a file is deleted by a keyboard monitor operation, a check is made if the file slot is the highest slot used, in which case the entire entry is zeroed out and the entry becomes the first of all available slots at the end of the directory. This generally prevents the CA{talog} command output from having trailing empty entries. However, there are perverse circumstances where a series of free slots may be seen ending the command output. This can always be eliminated by using the EN{ter} command to create dummy file entries and then deleting them last entry first as required. However, in doing so, attempts at recovery from accidental deletion will be hampered as all trailing characters in the file name are removed. Free slots are indicated by the value 7777 in the first of the three file name words; the other two are retained. This may aid in recovery from accidental file deletion since there may be meaningful artifact characters to help identify the file. Regardless of the circumstances leading to the free or available directory slot, a new file name can always be entered and the corresponding file can be loaded into the edit buffer to attempt recovery. More information on file recovery and many other subjects is available in the P?S/8 Keyboard Monitor Command Guide available as a separate document.

TCBIN is five characters, the trailing six bits in the name entry for TCBIN is 00. TCBOOT is six characters; the three words are a complete file name. Since FOO was deleted, the first two characters are replaced by the value 7777. The rest of the entry still retains 0 followed by three sets of six bits set to 00. BAR is six characters and active; three sets of six bits set to 00 fill the rest of the three word entry.

```

BLOCK: 0:0015

0000 2403 0211 1600 2403 0217 1724 7777 1700 *TCBIN@TCBOOT??O@*
0010 0000 0201 2200 0000 0000 0000 0000 0000 *@@BAR@@@@@@@@@@*
0020 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0030 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0040 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0050 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0060 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0070 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0100 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0110 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0120 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0130 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0140 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0150 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0160 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0170 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*

```

Figure 1: Dump of Block 0015 of P?S/8 Version 8Z for TC08.

Figure 2 shows the last three words of the TFS directory set to the specific values particular to the standard release of P?S/8 Version 8Z for the TC01/TC08 DECTape. System programs are allocated starting in very low blocks (after certain hard-wired data blocks reserved for various purposes including images of non-system device handlers). In the standard release of P?S/8 for TC01/TC08 DECTape, all system programs are stored in blocks ending at block 0735. (Additional blocks 0736 through block 0777 are reserved for future expansion of the system program area.)

word[177] of block 0016 contains the starting block for the first file entered into the TFS directory. In the stated example, this is block 1000. As such, the file TCBIN occupies blocks 1000-1017. The file TCBOOT occupies blocks 1020-1037. The deleted file (formerly known as FOO, but now the only information is that it was three characters and the third character is 0) occupies blocks 1040-1057. The file BAR occupies blocks 1060-1077. All blocks from 1100 through 2677 are unallocated as part of a series of available slots.

word[176] of block 0016 contains the high-limit block. The highest block of the final file allowed in the directory must end just prior to this block. In this example, the highest file allowed would occupy blocks 2660-2677.

word[175] was originally undefined in early releases of P?S/8; however this word was added to make the TFS structure somewhat more robust and allow elements of the keyboard monitor to be coded more easily. The information is essentially redundant to the high-limit block value in word[176] expressed in a more pragmatic manner. When the directory is full, attempts to access blocks as high (or higher) than the limit block are not allowed. This corresponds to the address within the TFS directory where the entry for the disallowed file would have been placed. The value stored in word[175] is this address expressed in two's complement form.

within the keyboard monitor, the image of the TFS directory is stored in 07000-07377. Instead of calculating the block of a tentative file and comparing to the high-limit block, this information is the address in memory of the first directory slot not allowed.

```

BLOCK: 0:0016
0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0010 0000 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0020 0000 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0030 0000 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0040 0000 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0050 0000 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0060 0000 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0070 0000 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0080 0000 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0090 0000 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0100 0000 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0110 0000 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0120 0000 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0130 0000 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0140 0000 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0150 0000 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0160 0000 0000 0000 0000 0000 0000 0000 0000 0000 *@@@@@@@@@@@@@@@@*
0170 0000 0000 0000 0000 0000 0514 2700 1000 *@@@@@@@@@ELW@H@*

```

Figure 2: Dump of Block 0016 of P?S/8 Version 8Z for TC08.

In the example given, the number of slots allowed would be $(2700-1000)/20$ or 74 files allowed. (Note: All examples are stated in terms of octal values; 74 octal files can also be expressed as 60 decimal files.)

Since each file entry is three words, 74×3 or 264 words of the directory can be used to represent files for this device. Since the directory is loaded into 07000 and onward, the first entry address disallowed would be 7264. The value in word[175] is expressed as the two's complement of the limiting address or 0514. Thus, if a TFS entry is attempted, it would be rejected if it was calculated as 7264 (or higher) within the directory address range of 07000-07377.

For larger devices, additional file slots are allowed, up to the maximum capacity of 124 (84 decimal) files occupying 2500 blocks. Extremely spartan DECTape-based logical units could permit this much file usage by eliminating virtually all system programs. As a practical example, TFS file storage can start at block 0200 (as of P?S/8 Version 8Z) by eliminating all system programs. The maximum capacity of the TFS directory will just fit on a standard DECTape of 2702 blocks; the files will be stored in blocks 0200-2677. (Note: Setting up a DECTape for use as a TFS files-only auxiliary tape can easily be accomplished; however, there are presently no standard utilities to automatically implement such a reconfiguration of resources. With a proper understanding of the TFS and system programs directory, this can easily be accomplished by an advanced user using either P?S/8 DUMP or P?S/8 BLKODT to implement the slight changes to the TFS and System Programs directories. Future releases of P?S/8 will include reconfiguration utilities to allow all users the ability to adjust file allocation storage as required.)

DECTapes are occasionally available with 3300 blocks; the maximum file capacity can be achieved by eliminating all system programs past block 0577. As of Version 8Z, this allows retention of *PAL*, *FOCAL* and most other system programs. Alternatively, the approach above could be used to allow additional storage beyond the TFS directory for use as the extended-length file on the device. (Note: Extended-length file support is currently undergoing redesign; documentation will be provided for future releases when the implementation is stable. The *P?S/8 Keyboard Monitor Command Guide* includes interim descriptions of how to take advantage of this structure as of Version 8Z.)

File block boundary considerations.

While there are no inherent requirements for any blocks within a TFS directory to be on any particular boundary, P?S/8 execution commands require that all files be aligned to boundaries of eight blocks. This restriction exists to allow P?S/8 to communicate the starting block and logical unit of any file in 12-bit words. With this restriction in place, the block number can be expressed in the high-order nine bits of the 12-bit word while the logical unit can be expressed in the low-order three bits of the same 12-bit word. Lists of such words are passed to system programs that will process the passed values accordingly.

Files from earlier systems.

Very early P?S/8 systems did not impose the boundary restriction stated above; the logical unit concept had not yet been implemented. It is not likely any of these early systems exist presently; however, the forerunner system known as the R-L Monitor System (or MS/8) for TC01/TC08 DEctape was submitted to DECUS and in theory was used by certain individuals who are likely candidates for upgrade to P?S/8. The R-L Monitor System imposes no boundary restrictions as logical units are not supported. (The R-L Monitor System is for a single TU55 or TU56H (half) drive on a TC01 or TC08 only; no other configuration is supported.)

The R-L Monitor System also uses the TFS directory format and is compatible with the P?S/8 Version 8Z keyboard monitor when used as a non-system logical device unit for file access (read purposes) only. Files in binary format can be copied directly as the R-L Monitor System binary file format is a compatible subset of that used for P?S/8 binary files (lacking support for extended memory). Text files require conversion using the P?S/8 stand-alone *RLFIX* utility (available from the author of this document).

It is recommended the R-L Monitor System DEctape be backed up before upgrading, as the *RLFIX* program modifies the supplied DEctape media; all converted text files will no longer be compatible with the R-L Monitor System; the conversion process is not reversible. The selected files will be upgraded to P?S/8 text format and stored back in-place with no loss of data. After the upgrade process has completed, all selected text (and binary) files will be fully compatible with P?S/8 standards. However, the TFS directory will not be in compliance with the boundary requirement needed for full access by P?S/8 programs.

As such, the files will require transfer to another P?S/8 device unit. (Note: The R-L Monitor System only supports TC01/TC08 DEctape as does the *RLFIX* program; as such, the conversion must generally be completed using the TC01/TC08 release of P?S/8 or perhaps a media-compatible TD8E-based P?S/8 release.) The file transfer can be accomplished with keyboard monitor commands such as `FE{tch} filename:1` followed by `WR{ite} filename:0` or similar operations. Alternatively, the P?S/8 *FILMAN* utility can be used to allow files to be copied in groups using the wild-card match feature of *FILMAN*; file types can be restricted at the same time if desired. (Additional information about P?S/8 *FILMAN* is available in other documents.)

Once the files have been upgraded and transferred to boundary-conforming media, the files can be used with all relevant P?S/8 utilities.

Note: All users are always encouraged to embellish text documents using horizontal tabs to maintain alignment. Unfortunately, The R-L Monitor System does not support horizontal tab as a valid text character. As such, users should consider additional editing of the files after conversion to P?S/8.

(end-of-file)